KB: A Hybrid-Retrieval Codebase Knowledge System with Multi-Objective Exploration Loss Evaluation

Jacob Rosenzweig

Fintary

jacob.rosenzweig@fintary.com

KB v2.0.1 • August 15, 2026

Abstract

LLM-based developer agents suffer from two compounding inefficiencies: each session begins with an empty context window, forcing costly re-exploration of the codebase, and current evaluation frameworks measure only whether the final answer is correct, hiding the exploration cost that produced it. We present two complementary contributions.

KB is a local-first codebase knowledge system that converts knowledge maintenance into a side-effect of normal development work. KB indexes a repository deterministically into three retrieval units—whole markdown *documents*, exported *code symbols* (via tree-sitter AST analysis), and curated *facts*—each stored in SQLite with a full-text (FTS5) index and an optional embedding vector computed at init. At query time, a *hybrid retriever* searches all three unit types through a lexical (BM25/FTS5) and a neural (embedding cosine) lane, fuses the six ranked lists with Reciprocal Rank Fusion, and pulls in related units through a single depth-1 hop over a flat document↔symbol link table—no graph walk, so retrieval cost is bounded by the candidate pool rather than by index connectivity. A judge-in-the-loop curator then drops off-topic units and refills reported gaps before one-shot synthesis returns plain prose.

MOEL (Multi-Objective Exploration Loss) is a quantitative evaluation framework that measures agent efficiency under three controlled conditions: Condition N (raw filesystem baseline), Condition K (KB-equipped), and Condition O (oracle context injection). MOEL combines three normalized loss terms—LLM jury semantic loss (L_{jury}), trajectory inefficiency ($L_{\text{trajectory}}$), and resource consumption (L_{resource})—into a single weighted scalar L_{MOEL} in a deep evaluation with toy-tool N/K/O conditions. Paired harvest evaluation instead compares `kb query` (K) to a real headless coding agent (N) via ΔS.

The headline metric is a budget-normalized success score $S = 0.60\,Q_{\text{adeq}} + 0.30\,E_{\text{tok}} + 0.10\,E_{\text{speed}}$ that blends adequacy-weighted answer quality (correctness, usefulness, and relevance with diminishing returns above τ=3) with token economy and latency, scored identically for KB-equipped and control runs. The project grade is $\Delta S = S_K - S_N$ from a single paired evaluation (Eq. 9). Harvest evaluations (August 15, 2026) cover ten upstream repositories (Table 2): kb self-check ($\Delta S = +0.267$, $S_K = 0.850$, $S_N = 0.583$), raylib ($\Delta S = +0.229$, $S_K = 0.832$, $S_N = 0.603$), and `fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, and `brew` (ΔS: +0.252, +0.133, +0.283, +0.311, +0.352, +0.097, +0.345, +0.139). paired K-vs-N evaluations on `kb`, `raylib`, `fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, `brew` (August 15, 2026).

1 Introduction

Large language models (LLMs) have dramatically changed how developers interact with codebases, enabling agents to autonomously resolve real-world software engineering issues [12]. Yet the dominant interaction model remains *ephemeral*: each agent session begins with an empty context window, forcing the model to re-discover facts about the codebase from scratch. This creates a compounding inefficiency—teams that rely on LLM assistants must accept that every question triggers a fresh, expensive, and error-prone tour of the filesystem.

Two failure modes characterize this status quo. First, **exploration waste**: agents read far more source files than necessary before reaching an answer, because there is no prior accumulation of structured knowledge to query. On representative SWE-Bench

tasks, text-based agents spend up to 21 steps navigating via grep and line-range reads to locate a single target function—work that structured pre-indexed facts resolve in two or three queries [6]. Second, **context dilution**: irrelevant context degrades LLM reasoning [9], and brittle string-matching edits fail when whitespace or formatting drifts [6].

Existing retrieval-augmented generation (RAG) systems [7] address the first problem partially, but are designed for static document corpora, not living codebases with interleaved code, documentation, and decision history. Code-search benchmarks such as CodeSearchNet [5] evaluate individual retrieval steps in isolation, ignoring the sequential cost of multi-step agent exploration.

We present **KB**, a local-first codebase knowledge system that:

(1) indexes a repository's documentation and code into a persistent SQLite store of three retrieval units—whole markdown documents, exported code symbols (via deterministic AST analysis), and curated facts—each with a full-text index and an optional embedding vector;

(2) exposes those units to LLM agents through a single-pass *hybrid retriever* that fuses six lexical (BM25) and neural (embedding similarity) lanes with Reciprocal Rank Fusion and adds a depth-1 document↔symbol hop, then a judge-in-the-loop *curator* that drops off-topic hits before one-shot synthesis—without manual query engineering.

Critically, KB's value is only as compelling as the evidence that it actually reduces agent exploration cost. Rubric-based Q&A scoring—the standard in prior agent evaluation—captures answer quality but is blind to *how* the agent arrived at that answer. A system that reads 40 files before synthesizing a correct response and one that queries two facts look identical under a pass/fail quality signal.

To fill this gap, we introduce the **Multi-Objective Exploration Loss (MOEL)** evaluation framework. Paired harvest evaluation scores both KB (K) and a real-agent control (N) on the same question pack per suite and reports the headline grade $\Delta S = S_K - S_N$ (Eq. 9). A complementary deep evaluation runs tasks under three conditions (N, K, O) and aggregates exploration loss into L_{MOEL}.

Figure 1 illustrates the end-to-end system. Our contributions are:

1. **KB**: a codebase knowledge system with deterministic document/symbol indexing, init-time per-unit embeddings for neural scoring, a single-pass hybrid rank-fusion retriever with a depth-1 document↔symbol hop, and a post-retrieval relevance curator that filters the unit pool before synthesis.

2. **MOEL**: a multi-objective evaluation framework comprising three normalized loss functions (L_{jury}, $L_{\text{trajectory}}$, L_{resource}), bias-mitigated LLM jury scoring, and logistic-regression calibration on a 20-sample human-annotated set.

3. **Empirical results**: paired harvest evaluation on ten fixed upstream snapshots—`kb` (self-check), `raylib`, `fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, and `brew`—comparing `kb query` (K) to a headless coding agent (N) under identical LLM scoring (Table 2). Headline grades: $\Delta S = +0.267$ (kb self-check) and $\Delta S = +0.229$ (raylib) as of August 15, 2026.

2 Related Work

2.1 LLM-Based Code Agents and Tools

SWE-Agent [12] navigates repositories through iterative file-read and string-edit operations. Agentless [11] takes a non-agentic approach—a fixed localization-then-repair pipeline that deliberately avoids giving the LLM autonomous tool-use—yet still requires direct file reads without pre-indexed context. AutoCodeRover [13] augments localization with AST-based code search APIs (e.g., `search_method_in_file`) that retrieve class and method implementations from a structured AST index. CODESTRUCT [6] replaces text-based edits with AST-grounded primitives (`readCode`/`editCode`), reducing input tokens by 12–38% and improving SWE-Bench Verified Pass@1 by up to 5.0pp. KB is complementary to these systems: whereas CODESTRUCT improves the *action interface* once a target is located, KB improves *context retrieval* by replacing filesystem exploration with structured fact lookup. The two approaches can be composed.

2.2 Retrieval-Augmented Generation

RAG [7] grounds LLM answers in retrieved passages, but standard RAG pipelines are designed for static document collections and typically retrieve a single unit type (text chunks) through a single lane. KB instead indexes a codebase as three complementary unit types—whole documents, exported code symbols, and curated facts—and fuses lexical (BM25)

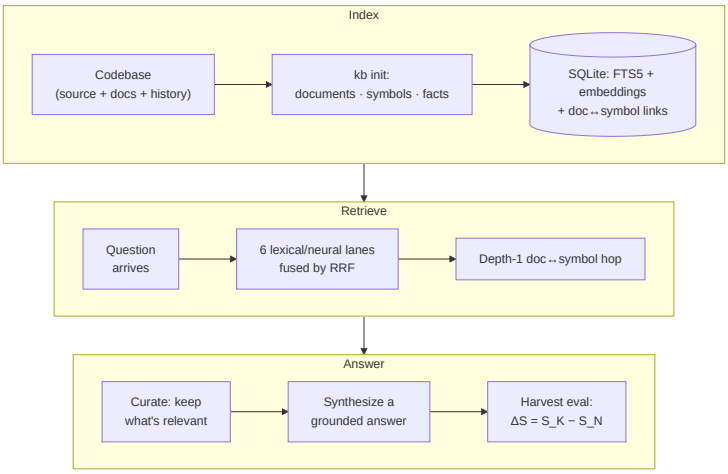

Figure 1: End-to-end KB system. Primary harvest grade: ΔS (K vs. real agent); deep MOEL evaluation adds L_{MOEL} under toy-tool N/K/O conditions.

and neural lanes over all three with Reciprocal Rank Fusion [2], then follows a depth-1 document↔symbol link so an answer can join prose to the code it describes. This captures the doc/code correspondence that flat chunk RAG misses while keeping retrieval a single bounded pass rather than an unbounded graph traversal.

2.3 LLM Evaluation Frameworks

Evaluating LLM outputs with another LLM (LLM-as-judge) is established in MT-Bench [14], which identifies position bias, verbosity bias, and self-enhancement bias as the primary failure modes. AlpacaFarm [3] applies a related idea—using LLMs to simulate human preference feedback—for RLHF training at lower cost. PandaLM [10] trains a dedicated judge model for reliable instruction-tuning evaluation. MOEL's jury module addresses the same biases at inference time through position debiasing (swapped-order consistency), verbosity normalization ($\log_{1+}$ length scaling), self-enhancement down-weighting ($\times 0.5$ for same-family judges), and family diversity enforcement—without requiring a fine-tuned judge. Unlike prior work, MOEL is not outcome-only: deep MOEL evaluation combines the LLM jury with trajectory and resource losses so that efficiency failures are numerically visible; paired harvest evaluation adds ΔS against a real agent baseline.

2.4 Agent Efficiency Measurement

SWE-ContextBench measures token cost and cache efficiency across context injection strategies; CodeScaleBench evaluates tool validity across repository scale classes. Neither framework provides a unified scalar loss that spans correctness, trajectory, and resource consumption simultaneously. The closest prior work is SWE-Atlas, which verifies agent-declared file manifests against live repository diff output. MOEL's `ManifestValidator` adopts the same live-diff approach and extends it with a `MutationValidator` that applies tree-sitter AST body stubs to confirm causal linkage between agent changes and test coverage.

2.5 AST-Based Code Analysis

Tree-sitter and GumTree [4] provide fine-grained AST differencing and incremental parsing. code2vec [1] uses AST path embeddings for code representation learning. KB's code facts are derived deterministically from tree-sitter exports; MOEL's programmatic validators (manifest and mutation checks) extend SWE-Atlas-style live-diff verification with AST-grounded causal linkage.

3 KB: A Hybrid-Retrieval Codebase Knowledge System

3.1 Knowledge Representation

KB represents a codebase as three kinds of independently retrievable *units* stored in a SQLite database, rather than as a single graph of atomic facts:

- **Documents** — whole markdown files (README, docs) stored verbatim in a `documents` table (`title`, `body`, `content_hash`, origin `git_repo`); the retrieval unit is the entire document, not a sentence chunk.

- **Code symbols** — exported declarations (functions, classes, interfaces, type aliases, constants) extracted by AST analysis and stored in a `code_symbols` table with their name, `kind`, and source text.
- **Facts** — curated natural-language claims in a `facts` table, retained for authoring provenance and as a third retrieval lane.

Each unit type has a paired FTS5 full-text index (`documents_fts`, `code_symbols_fts`, `facts_fts`) and an optional per-unit embedding vector (`doc_embeddings`, `code_embeddings`, and fact embeddings). Documents and symbols are associated through a flat `doc_code_links` table—a document is linked to the symbols it describes. This is deliberately *not* a traversable graph: there are no typed fact edges and no multi-hop neighborhoods. An earlier version of KB stored sentence-level facts in a `fact_edges` graph and walked it at query time; that graph was removed in favor of the flat, depth-1 link table because unbounded neighborhood expansion made retrieval cost scale with index connectivity rather than with the query.

3.2 Indexing Pipeline

`kb init` executes a fully deterministic indexing pipeline. No LLM is invoked and no API cost is incurred.

Code symbols. For each source file, KB invokes the `web-tree-sitter` WASM runtime with per-language export queries to extract named declarations. Each exported symbol becomes a `code_symbols` row keyed by `(git_repo, rel_path, name)`, storing its kind and source text for later display and neural scoring.

Documents. Human-readable sources (README files, markdown documentation) are ingested *whole*: each file becomes one `documents` row with a content hash for incremental re-indexing. Open Knowledge Format frontmatter is stripped so metadata never pollutes the body. KB then records `doc_code_links` between a document and the symbols it references.

Embeddings. After indexing completes, `kb init` runs a best-effort embedding pass over documents and symbols: by default a local on-device sentence-transformer (`all-MiniLM-L6-v2`); set `KB_EMBEDDER=gemini` to opt into hosted Gemini embeddings. When no embedder is available, rows keep a deterministic hash vector and neural scoring degrades gracefully—init never blocks on embedding failure, and the lexical lanes carry retrieval on their own.

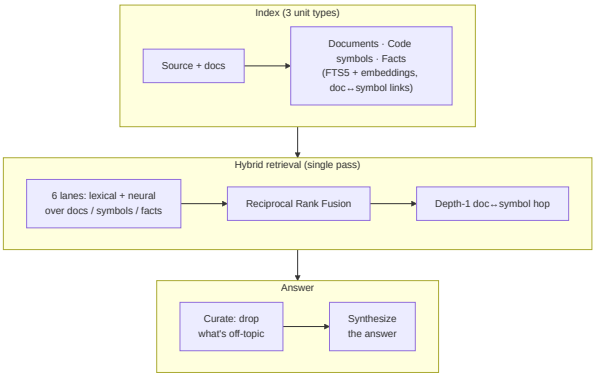

Figure 2: KB indexing (top) feeds the hybrid retriever: six lexical/neural lanes over documents, symbols, and facts are fused by Reciprocal Rank Fusion, a depth-1 doc↔symbol hop enriches the pool, and a curator prunes before one-shot synthesis (bottom).

The raylib snapshot in Table 2 (0 entities, 12,309 links) completes init+scan in minutes on a laptop with no LLM indexing cost.

3.3 Retrieval: Hybrid Rank Fusion

At query time, KB runs a single-pass *hybrid* retriever over the three unit types. For a query q it opens six ranked lanes—each of {documents, code symbols, facts} searched both lexically (BM25 [8] over FTS5) and neurally (embedding cosine). To avoid a full-table embedding scan per query, the neural lanes rerank the candidate pool that the lexical lanes surfaced rather than scoring every vector in the index.

Reciprocal Rank Fusion. The six lists are combined with Reciprocal Rank Fusion (RRF) [2]: a unit at rank i in a lane contributes $1/(k+i)$ to its fused score, with damping $k = 60$. RRF needs no per-lane score calibration—it consumes only ranks—so lexical BM25 scores and cosine similarities combine without a tuned weighting between them, and a unit surfaced by several lanes accrues their contributions.

Depth-1 enrichment hop. After fusion, KB takes the top few documents and symbols and follows `doc_code_links` exactly one hop: the code a top document describes, and the documentation that describes a top symbol. Hopped units enter *below* every directly matched unit, so they enrich the pool without displacing direct hits. Because the link table is flat and the hop is depth-1 by construction, retrieval cost is bounded by the candidate pool, not by how densely the index is connected—the property the removed graph walk lacked. The top ~ 40 fused units are handed to curation and synthesis.

3.4 Post-Retrieval Curation

Rank fusion and the enrichment hop are deliberately *broad*: they surface units adjacent to, but not always directly about, the user's question. When the ranked pool is large enough, a **curator** runs *before* synthesis. It is judge-in-the-loop, not a static score floor:

1. **Deterministic auto-keep**: units whose token overlap with the *raw user question* exceeds a fixed threshold bypass the LLM (no cost, stable anchors).

2. **Candidate cap**: only the top-ranked units enter the LLM verdict; the tail is dropped deterministically so large pools cannot overflow the structured keep/drop JSON.

3. **Structured relevance verdict**: remaining candidates are sent to a single LLM call returning a structured verdict with *keep* ids, *gap* descriptions, and a *sufficient* flag.

4. **Drop**: every unit not in the keep set (except auto-kept rows) is removed from the synthesis context; there is no minimum keep fraction.

5. **Gap re-discovery**: when the judge reports gaps and the set is not yet sufficient, the curator issues bounded hybrid searches per gap (excluding already-seen ids), merges new hits, and re-judges for up to a small round cap.

On LLM or parse failure the curator *fails safe* to the incoming pool for small sets, and caps at the top-ranked subset for larger pools rather than passing an unfiltered pool into synthesis. Curator audit counts (kept, dropped, re-queried) are recorded separately from the synthesis context for later analysis—never injected back into the prompt as evidence.

3.5 Answer synthesis

After retrieval and optional curation, `kb query` performs *one-shot* synthesis: a single LLM call over the curator-kept units (document bodies are truncated to a per-unit character cap). The synthesis prompt instructs *plain prose*: no inline "(fact n)" citations and no trailing Sources/Citations block; provenance is surfaced in a separate structured evidence summary. This is the path evaluated against the control agent in §5.5—no multi-turn retrieval loop, matching how an external agent would call KB for a standalone question.

4 MOEL: Multi-Objective Exploration Loss

4.1 Motivation

Standard evaluation of LLM agents on code tasks measures whether the final answer is correct. This binary signal hides the cost of exploration: an agent that issues 50 tool calls to reach a correct answer and one that issues 3 are indistinguishable under pass/fail scoring. Similarly, LLM-as-judge evaluators have known failure modes (position bias, verbosity inflation, self-enhancement) that inflate quality estimates when judges are not calibrated [14].

MOEL addresses both problems by combining three normalized loss components into a single scalar, evaluated under three controlled conditions per task.

4.2 Three-Condition Evaluation Design

The *deep* MOEL evaluation runs each task under three toy-tool conditions (Table 1). This differs from the *harvest* evaluation (§5), where condition N is a headless coding agent with filesystem exploration tools and condition K is `kb query`—the pair that defines ΔS.

Table 1: Deep MOEL evaluation: toy-tool conditions per task (not the harvest N/K pair).

Cond.	Tools available	Purpose
N (raw FS)	`read_file`, `list_directory`, `search_file_contents`	Worst-case baseline
K (KB)	Full KB retrieval and action tools	Primary experiment
O (Oracle)	Minimal facts injected in system prompt	Ceiling

The primary hypothesis is $L_{\text{MOEL}}(N) > L_{\text{MOEL}}(K)$; secondary goal is $L_{\text{MOEL}}(K) \approx L_{\text{MOEL}}(O)$ as the knowledge base matures. Harvest headline results (§5.5) use ΔS instead.

4.3 Loss Functions

L_{jury} — LLM Jury Semantic Loss

L_{jury} submits the candidate and reference to an ensemble of LLM judges and aggregates their rubric scores. Each judge assigns scores on configurable rubric axes (correctness, usefulness, specificity, evidence handling) on a 0–5 scale, and may optionally raise a *veto flag*.

Bias mitigation. Four debiasing mechanisms are applied:

1. **Verbosity normalization**: a $\log_{1+}$ length-scaling term is added to the rubric, penalizing unnecessary verbosity without penalizing accurate, detailed responses.

2. **Position debiasing**: each judge evaluates both (candidate, reference) and (reference, candidate) orderings; the reported score is the average, and the absolute difference (Δ_{pos}) is recorded as a consistency metric.

3. **Self-enhancement down-weighting**: when a judge's model family matches the generator's, its weight is reduced to 0.5.

4. **Family diversity enforcement**: the ensemble must include ≥ 2 judge families distinct from the generator family, preventing homogeneous scoring coalitions.

Minority-veto policy. If ≥ 2 judges veto, $L_{\text{jury}} = 1.0$; otherwise it equals $\bar{s}_w$, the weighted mean of per-judge normalized scores:

$$L_{\text{jury}} = \begin{cases} 1.0 & \text{veto count} \geq 2 \\ \bar{s}_w & \text{otherwise} \end{cases} \tag{1}$$

Statistical calibration. Jury scores are calibrated using per-judge logistic regression:

$$P(\text{correct} \mid s) = \sigma(\beta_0 + \beta_1 s) \tag{2}$$

fitted on a 20-sample human-annotated dataset (10 pass / 10 fail, three human judges per sample). Generalization error is estimated via leave-one-out cross-validation. The pipeline is invoked once per judge model and stores (β_0, β_1) coefficients for use at scoring time.

$L_{\text{trajectory}}$ — Trajectory Inefficiency Loss

$L_{\text{trajectory}}$ penalizes two failure modes: taking more steps than a configured ceiling, and repeating identical tool calls. Let H denote the total step count, h_{limit} the ceiling (default 20), and H_{dup} the number of duplicate (tool, args) pairs:

$$L_{\text{traj}} = \min\left(\tfrac{1}{2}\min\left(\tfrac{H}{h_{\text{lim}}}, 1\right) + \tfrac{1}{2}\,\tfrac{H_{\text{dup}}}{H},\ 1\right) \tag{3}$$

Duplicate detection normalizes argument keys (alphabetical sort, whitespace trim) so that logically identical calls with different JSON serializations are correctly collapsed.

L_{resource} — Resource Consumption Loss

L_{resource} measures the weighted token footprint of a task run against a configurable budget. Following Anthropic's prompt caching pricing, cached tokens are discounted by $\delta = 0.1$, and output tokens are weighted at $\gamma = 1.0$:

$$L_{\text{resource}} = \min\left(\frac{C_{\text{fresh}} + \delta \cdot C_{\text{cached}} + \gamma \cdot C_{\text{output}}}{\text{budget}},\ 1\right) \tag{4}$$

Token counts are drawn from per-step records that distinguish fresh, cached, and output tokens; aggregated totals that lack this split are not used.

4.4 MOEL Aggregation

The three components combine into a single scalar:

$$L_{\text{MOEL}} = w_C \cdot L_{\text{jury}} + w_T \cdot L_{\text{traj}} + w_R \cdot L_{\text{res}} \tag{5}$$

Default weights: $w_C = 0.5$, $w_T = 0.3$, $w_R = 0.2$. The constraint $w_C + w_T + w_R = 1$ is validated to within 10^{-6} and all loss terms are bounded to $[0, 1]$.

Harvest adequacy scoring. The headline harvest evaluation (§5.4) uses a complementary *gain* formulation S rather than L_{MOEL}, but shares the same design principle: quality is measured through an adequacy utility φ_τ (Eq. 6) that treats τ=3 on the 0–4 rubric as "good enough" and applies diminishing returns above it, so token and latency savings are not drowned out by marginal rubric polish.

4.5 Benchmark Alignment

MOEL's three-condition design (N/K/O) mirrors SWE-ContextBench's context injection conditions; L_{resource} replicates that benchmark's token-cost metric with the additional fresh/cached split. CodeScaleBench's tool-validity tier corresponds to MOEL's condition comparison: Condition N uses only primitive filesystem tools, Condition K adds the full KB tool set.

5 Experiments

5.1 Evaluation Targets

Harvest evaluation uses a fixed portfolio of ten upstream repositories (Table 2). Each suite is a version-pinned git snapshot with a curated question pack scored by the same LLM-as-judge rubric.

Raylib (external benchmark). The raylib C library is the primary *external* target: it is not the KB codebase (eliminating evaluator familiarity bias),

has rich cross-module structure that exercises the document↔symbol hop, and ships stable upstream documentation. $N_q = 11$.

KB self-check. Suite `kb` evaluates KB against its own codebase—twelve architecture, workflow, and adversarial questions (including fact-curator and out-of-scope probes). This smoke test stress-tests retrieval grounding on a repo whose structure the authors know well.

Cross-language portfolio. Eight additional suites stress parsers, retrieval, and product/UI inference across languages and architectural styles: `fzf` (Rust CLI fuzzy finder), `kestra` (Java orchestration with YAML flows and execute console), `shellcheck` (Haskell monadic AST linting), `lazygit` (Go TUI), `datasette` (Python data-explore/publish UI and CLI), `mitmproxy` (Python async proxy and TUI), `fish-shell` (Rust systems shell), and `brew` (Ruby package-manager DSL). Each uses $N_q = 8$ multi-step questions spanning capabilities, architecture, installation, conventions, and gotchas.

5.2 Protocol: paired K vs N evaluation

Each evaluation executes, in order: (1) clone and index the suite repository (`kb init` on first use, or reuse the named session and `kb scan` on subsequent runs; `--force-init` deletes the base and re-runs `kb init` from scratch); (2) N_q `kb query` calls (condition **K**)—one-shot synthesis over the curator-kept unit pool (§3.5); (3) the same questions handed to a headless coding agent with no KB (condition **N**); (4) LLM scoring (×3 averaged) on the five quality axes for both sides, plus measured token and latency totals; Question counts vary by suite ($N_q = 12$ for `kb`; $N_q = 11$ for `raylib`; $N_q = 8$ for the remaining suites).

Both sides are scored under identical rubric and success-score formulas; the headline grade is $\Delta S = S_K - S_N$ (Eq. 9).

5.3 Scoring Rubric

Each suite covers questions spanning capabilities, architecture, installation, configuration, platform specifics, conventions, gotchas, and (for `kb`) adversarial or out-of-scope probes—topics that stress different lanes of the hybrid retriever (documents, symbols, and facts). Scoring has two families that both enter the harvest grade S (Eq. 7):

Axis	Criterion / measurement
LLM-judged quality (ordinal labels 0–4)	
Correctness	Factually grounded in the repo/KB
Usefulness	Helps a developer act or understand
Relevance	Stays on the question; free of unrelated facts
Specificity	Uses concrete repo-specific details
Evidence Handling	Constrained to evidence; acknowledges uncertainty
Measured efficiency (numeric telemetry)	
Token usage	Weighted T (in+out; N adds 0.1×cache_read)
Latency	Wall-clock duration τ_w

How axes feed S. Correctness, usefulness, and relevance enter the adequacy term Q_{adeq} via φ_τ (Eq. 6). Token usage and latency become the efficiency terms E_{tok} and E_{speed} (Eq. 8). Specificity and evidence handling are scored and reported as diagnostics but do not enter Q_{adeq} or S. The secondary pass rate counts questions where correctness, usefulness, *and* relevance are all ≥ 3.

Ordinal labels, not bare numbers. Rather than ask the LLM judge for "an integer 0–4"—which invites guessing a point on an unanchored scale—each quality axis is scored by selecting one of five *named ordinal labels*. Correctness, for example, ranges over *correct* (4), *mostly_correct* (3), *mixed* (2), *mostly_wrong* (1), and *no_answer* (0); the other quality axes use analogous per-axis vocabularies (e.g. relevance: *focused/on_topic/padded/off_topic/unrelated*). This reframes scoring as classification—"which description fits this answer?"—rather than magnitude estimation, which is markedly more reliable for LLM judges. Each label maps deterministically to its ordinal level $s \in \{0, 1, 2, 3, 4\}$, so the adequacy utility (Eq. 6) and every downstream metric operate on the same 0–4 scale as before and historical runs remain comparable. Token usage and latency are measured from run telemetry and are never labeled.

5.4 Success Score and Headline Verdict (ΔS)

The paired harvest evaluation's single scalar per side is $S \in [0, 1]$ (higher is better). It trades *adequate* answer quality against operational cost: we care that responses are correct and meaningful, but rubric scores above an acceptability threshold τ yield diminishing marginal value—polishing a 3.5 into a 4.0 should not dominate token or latency savings.

Adequacy utility. Each quality axis $s \in [0, 4]$ (correctness, usefulness, relevance) maps to a normalized utility with threshold $\tau = 3$ and excellence discount $\beta = 0.2$:

$$\varphi_\tau(s) = \frac{1}{1+\beta}\left(\min\Big(1,\ \frac{s}{\tau}\Big) + \beta \cdot \frac{\max(0,\ s-\tau)}{4-\tau}\right) \tag{6}$$

Below τ, quality rises linearly (inadequate answers are heavily penalized). At $s = \tau$, $\varphi_\tau = 1/(1+\beta) \approx 0.83$; at $s = 4$, $\varphi_\tau = 1$. The gap from "acceptable" to "perfect" is therefore $\approx 17\%$ of the quality scale—not the 33% implied by linear $(c+u)/8$ scoring.

Composite score. The harvest success score blends adequacy quality with token economy and speed:

$$S = w_Q \cdot Q_{\text{adeq}} + w_T \cdot E_{\text{tok}} + w_V \cdot E_{\text{speed}} \tag{7}$$

with default weights $w_Q = 0.60$, $w_T = 0.30$, $w_V = 0.10$:

$$\begin{aligned} Q_{\text{adeq}} &= \tfrac{1}{3}\big(\varphi_\tau(\bar{c}) + \varphi_\tau(\bar{u}) + \varphi_\tau(\bar{r})\big), \\ E_{\text{tok}} &= 1 - \min\Big(\tfrac{T}{B_T},\ 1\Big), \\ E_{\text{speed}} &= 1 - \min\Big(\tfrac{\tau_w}{B_\tau},\ 1\Big) \end{aligned} \tag{8}$$

where $\bar{c}, \bar{u}, \bar{r}$ are mean correctness, usefulness, and relevance (0–4), so an answer that drags in unrelated facts is penalized even when correct and useful, T is the weighted token total for the question side (input + output + 0.1 × cache_read for control agents; kb query uses undifferentiated input+output), and τ_w is wall-clock latency. Budgets $B_T = 10^6$ tokens and $B_\tau = 6 \times 10^5$ ms are fixed (budget-absolute). KB (K) and control (N) use the identical formula.

Headline project grade. The single number that answers *does KB beat a real coding agent?* is the success-score *difference* in the same evaluation run:

$$\Delta S = S_K - S_N \tag{9}$$

$\Delta S \geq +0.02 \Rightarrow$ KB **ahead of control**; $\Delta S \leq -0.02 \Rightarrow$ **behind**; otherwise **on par**. Per-side scores S_K and S_N come from Eq. 7; their component parts (Q_{adeq}, E_{tok}, E_{speed}) show *where* KB wins or loses (e.g. adequate quality at lower exploration cost).

Secondary diagnostics. Pass rate and per-axis rubric means (correctness, usefulness, relevance, etc.) help explain a ΔS outcome but are not the headline grade. We also report the fraction of questions where correctness, usefulness, *and* relevance all score ≥ 3 on the 0–4 scale.

Retrieval-side relevance. For condition K, we log how many retrieved units the curator kept, dropped, and re-queried (§3.4), and compute retrieval precision as kept/(kept + dropped). This complements answer-level relevance in the rubric: low precision with high answer relevance indicates heavy pre-synthesis filtering; low answer relevance despite curation signals synthesis or gap-fill gaps.

Scoring stability. LLM-as-judge scoring is nondeterministic even at temperature 0; we average three scorer calls per question.

5.5 Results: harvest K vs N

Table 2 reports paired harvest results (August 15, 2026). paired K-vs-N evaluations on `kb`, `raylib`, `fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, `brew` (August 15, 2026).

Interpretation. On kb self-check (12 questions), $\Delta S = +0.267$ with $S_K = 0.850$ and $S_N = 0.583$; K completes the query side in 161s using 319,441 weighted tokens vs. N at 1841s and 1,743,770 tokens (mean relevance on K: 3.727; pass rate on three quality axes: 0.955). N leads on adequacy (Q_{adeq}: 0.972 vs. 0.954); K's ΔS margin comes mainly from token economy (E_{tok}: 0.681 vs. 0.000) and wall-clock speed (E_{speed}: 0.732 vs. 0.000).

On raylib, $\Delta S = +0.229$ with $S_K = 0.832$ and $S_N = 0.603$; K is faster (93s vs. 949s) while N leads on adequacy (Q_{adeq}: 1.000 vs. 0.813). Mean relevance on K is 3.462 (N: 4.000); pass rate 0.615. K uses 133,257 weighted tokens vs. N's 991,547 (E_{tok}: 0.867 vs. 0.008).

Portfolio suites. The remaining eight repositories report paired ΔS values of `fzf` +0.252, `kestra` +0.133, `shellcheck` +0.283, `lazygit` +0.311, `datasette` +0.352, `mitmproxy` +0.097, `fish-shell` +0.345, and `brew` +0.139 (S_K and pass rates in Table 2). They span Rust CLI tooling, Java/UI orchestration, Haskell AST analysis, Go and Python TUIs, Python data products, proxy/networking code, and Ruby DSL packaging—exercising parsers, hybrid retrieval, and UI/product inference beyond monolithic C libraries. Control agents vary by suite: `kb`, `raylib`, `fzf`, `shellcheck`, `lazygit`, and `fish-shell` use Cursor Agent (`composer-2.5`); `kestra`, `datasette`, `mitmproxy`, and `brew` use Claude Code (`claude-sonnet-5`). Each ΔS is a within-suite paired comparison; cross-suite N-side comparisons should treat the agent backend as a confound.

Table 2: Harvest evaluation results (August 15, 2026; LLM-as-judge, averaged over three scorer calls). Ten version-pinned upstream suites; each row pair reports K (`kb query`) and N (headless coding agent) under identical scoring. Headline grade $\Delta S = S_K - S_N$.

Suite	Side	S	Q_{adeq}	E_{tok}	E_{speed}	Pass	Corr	Rel	Tokens	Time (s)
kb self-check 2c6ea42	K (kb query)	0.850	0.954	0.681	0.732	0.955	3.714	3.727	319,441	161
	N (Headless agent: cursor-agent (composer-2.5))	0.583	0.972	0.000	0.000	0.909	3.773	3.864	1,743,770	1841
raylib dad422a	K (kb query)	0.832	0.813	0.867	0.845	0.615	2.638	3.462	133,257	93
	N (Headless agent: cursor-agent (composer-2.5))	0.603	1.000	0.008	0.000	1.000	4.000	4.000	991,547	949
fzf bd4efa2	K (kb query)	0.923	0.963	0.873	0.835	1.000	3.670	4.000	127,441	99
	N (Headless agent: cursor-agent (composer-2.5))	0.671	1.000	0.236	0.000	1.000	4.000	4.000	763,921	1057
kestra 562ebcb	K (kb query)	0.942	0.983	0.885	0.867	1.000	3.900	3.900	114,515	80
	N (Headless agent: claude-code (claude-sonnet-5))	0.809	1.000	0.657	0.119	1.000	4.000	4.000	343,158	529
shellcheck 9af7ee2	K (kb query)	0.915	0.933	0.899	0.857	1.000	3.400	3.900	101,358	86
	N (Headless agent: cursor-agent (composer-2.5))	0.632	1.000	0.107	0.000	1.000	4.000	4.000	892,518	1306
lazygit 6032225	K (kb query)	0.926	0.961	0.881	0.854	1.000	3.700	3.900	118,542	87
	N (Headless agent: cursor-agent (composer-2.5))	0.615	1.000	0.050	0.000	1.000	4.000	4.000	950,247	617
datasette 0337fba	K (kb query)	0.948	1.000	0.884	0.833	1.000	4.000	4.000	116,160	100
	N (Headless agent: cursor-agent (composer-2.5))	0.596	0.994	0.000	0.000	1.000	3.900	4.000	1,228,930	1012
mitmproxy bae1a7e	K (kb query)	0.925	0.956	0.892	0.841	0.900	3.600	3.900	107,785	96
	N (Headless agent: claude-code (claude-sonnet-5))	0.828	1.000	0.758	0.000	1.000	4.000	4.000	241,588	656
fish-shell 88db09e	K (kb query)	0.945	0.989	0.892	0.844	1.000	3.900	4.000	108,103	94
	N (Headless agent: cursor-agent (composer-2.5))	0.600	1.000	0.000	0.000	1.000	4.000	4.000	1,244,943	1049
brew ff4c18d	K (kb query)	0.922	0.972	0.855	0.824	1.000	3.830	3.830	144,910	106
	N (Headless agent: claude-code (claude-sonnet-5))	0.783	0.931	0.749	0.000	0.875	3.500	3.625	250,795	605

Quality vs. efficiency trade-off. Across the portfolio the control agent typically leads on adequacy (Q_{adeq}) and mean rubric scores, while K trails on answer quality (e.g. kb self-check correctness 3.714, raylib 2.638). K nonetheless wins ΔS on every suite because the success score weights token economy and latency (w_T=0.30, w_V=0.10): K's weighted token totals are an order of magnitude smaller than N's, and K completes the query phase faster. The adequacy gap remains the main headroom for future retrieval and curation work; under the fixed S weights it does not overturn the efficiency margin.

Retrieval cost profile. The hybrid retriever is a *single bounded pass*: the six lexical/neural lanes and the depth-1 document↔symbol hop are deterministic SQLite work with no LLM in the loop, so—unlike the iterative graph-walk it replaced—retrieval no longer consumes model tokens by exploring until an in-loop judge declares the pool sufficient. On the K side the only LLM calls are the curator's structured relevance verdict (plus any bounded gap re-queries) and the one-shot synthesis call; per-question telemetry records the thinking-vs-synthesis token split and the curator's kept/dropped/re-queried counts (retrieval precision, Table 2). Removing the unbounded loop is what keeps K-side weighted token totals an order of magnitude below the control agent's while retrieval cost stays bounded by the candidate pool rather than by index connectivity.

6 Conclusion

We introduced KB, a local-first codebase knowledge system. KB's deterministic document/symbol indexing and single-pass hybrid rank-fusion retrieval provide agents with structured, persistent codebase knowledge without per-session re-exploration. A post-retrieval curator enforces relevance on the fused unit pool before `kb query` uses one-shot synthesis to produce plain prose answers. Init-time per-unit embeddings (local by default) feed the neural retrieval lanes. Paired harvest evaluations (August 15, 2026) report $\Delta S = +0.267$ on the `kb` self-check suite and $\Delta S = +0.229$ on raylib; the full ten-suite portfolio (`fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, `brew`; Table 2) shows K ahead of control on every suite under the fixed S weights, despite N leading on adequacy. paired K-vs-N evaluations on `kb`, `raylib`, `fzf`, `kestra`, `shellcheck`, `lazygit`, `datasette`, `mitmproxy`, `fish-shell`, `brew` (August 15, 2026).

We also introduced MOEL, a multi-objective evaluation framework spanning two settings: (1) paired

harvest evaluation, whose headline grade is $\Delta S = S_K - S_N$ from runs comparing `kb query` to a headless coding agent on the same suite questions; and (2) deep MOEL evaluation (L_{MOEL}), which measures exploration loss under toy-tool N/K/O conditions per task. Bias-mitigated jury ensemble, statistical calibration, and programmatic validators distinguish efficient exploration from expensive correctness. Future work includes closing the adequacy gap on portfolio suites while preserving K's token and latency advantages, and extending deep MOEL coverage beyond the current task set.

References

[1] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[3] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, et al. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014.

[5] Hamel Husain, Ho-Howard Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019.

[6] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2025.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petrov, Vladimir Karpukhin, Yinfei Yang, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[9] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[10] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, et al. PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization. *arXiv preprint arXiv:2306.05087*, 2023.

[11] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024.

[12] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[13] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.